UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE



TEMA: Integridad – Bcrypt

ASIGNATURA: Ing. Seguridad Del Software

DOCENTE: Ing. Pablo Israel Morales Paredes

ESTUDIANTE: Emily Anabel Panchi Panchi

IMPLEMENTACIÓN DE AUTENTICACIÓN CON FLASK, BCRYPT Y MARIADB

1. Introducción

En esta práctica desarrollé un Portal Estudiantil con las funciones de registro e inicio de sesión. Para proteger las contraseñas utilicé bcrypt, que genera un hash antes de guardar la información del usuario. De esta manera, la base de datos no conserva la contraseña original.

Primero construí y probé la aplicación con Flask y SQLite en el servidor MASTER. Más adelante conecté el proyecto con MariaDB, porque necesitaba consultar los registros desde DBeaver sin copiar archivos de base de datos entre las máquinas. La aplicación quedó alojada en MASTER y realicé las comprobaciones desde Kali Linux.

En el desarrollo presento primero una vista del resultado final en DBeaver, para que se entienda qué se logró. Después explico las actividades en el orden en que fueron realizadas. Las capturas corresponden al servidor MASTER o a la aplicación y la base de datos de MASTER vistas desde Kali.

2. Objetivos

2.1. Objetivo general

Implementar un sistema de registro e inicio de sesión con Python y Flask que proteja las contraseñas mediante bcrypt y almacene los usuarios en una base de datos verificable desde una herramienta gráfica.

2.2. Objetivos específicos

- Comprobar la conectividad entre Kali Linux y el servidor MASTER y preparar las herramientas necesarias para ejecutar la aplicación.

- Desarrollar y probar el registro, el inicio de sesión y la validación de credenciales utilizando Flask y bcrypt.
- Crear la tabla de usuarios en SQLite y posteriormente en MariaDB para comprobar desde DBeaver que se almacenan los registros y sus hashes.

3. Desarrollo de la práctica

El desarrollo siguió dos momentos: primero desarrollé el Portal Estudiantil con SQLite en Master; finalmente cambié el almacenamiento a MariaDB en Master y consulté sus registros con DBeaver desde Kali. Las capturas se presentan siguiendo ese orden para identificar claramente dónde realicé cada actividad.

3.1. Vista general del resultado en DBeaver

Desde Kali abrí la base portal_estudiantil alojada en MASTER. DBeaver mostró cuatro registros en la tabla usuarios y sus valores en password_hash. Los hashes visibles comienzan con el formato habitual de bcrypt; no se almacenan las contraseñas en texto legible.

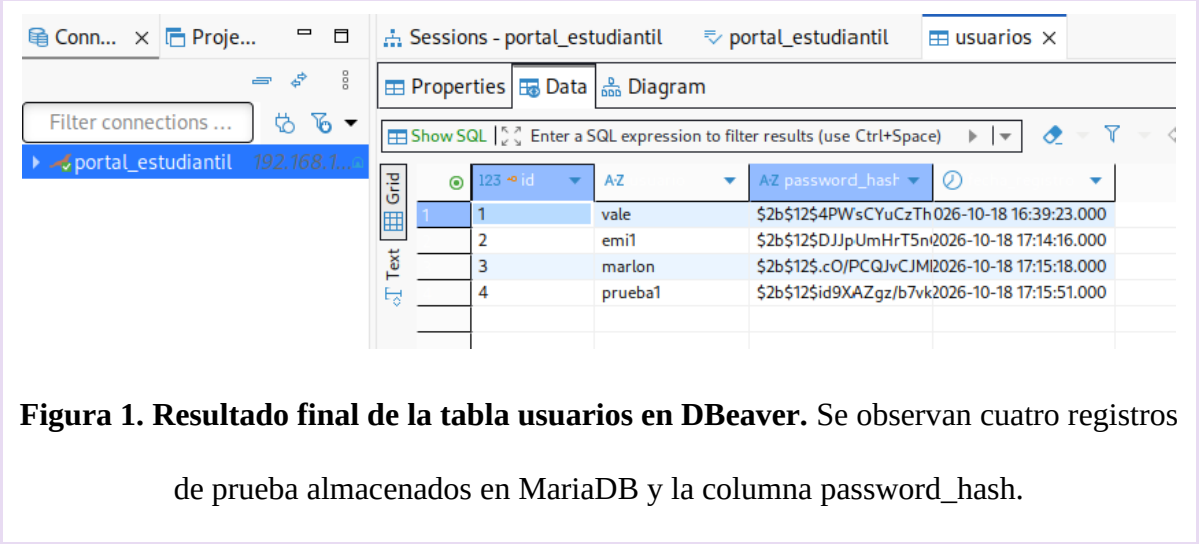


Figura 1. Resultado final de la tabla usuarios en DBeaver. Se observan cuatro registros de prueba almacenados en MariaDB y la columna password_hash.

Para consultar esa tabla configuré una conexión MariaDB en DBeaver con la base portal_estudiantil y un usuario de lectura. El acceso al servidor se realizó a través de SSH.

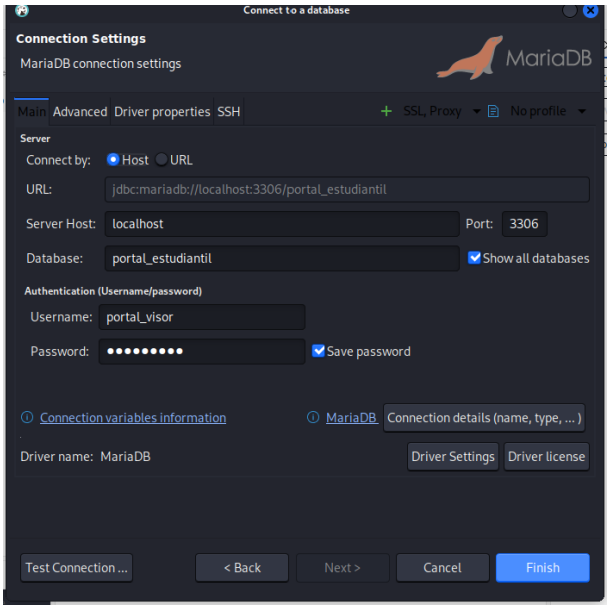


Figura 2. Configuración de la conexión MariaDB en DBeaver. La ventana presenta la base portal_estudiantil y la cuenta portal_visor utilizada para consultar los datos.

3.2. Preparación del servidor MASTER y primera etapa con SQLite

3.2.1. Conectividad entre Kali y MASTER

Comencé comprobando que Kali se comunicara con MASTER, cuya dirección en el laboratorio fue 192.168.100.165. La prueba respondió sin pérdida de paquetes. Después ingresé al servidor por SSH para realizar la configuración desde la terminal.

Comando utilizado

ping -c 4 192.168.100.165

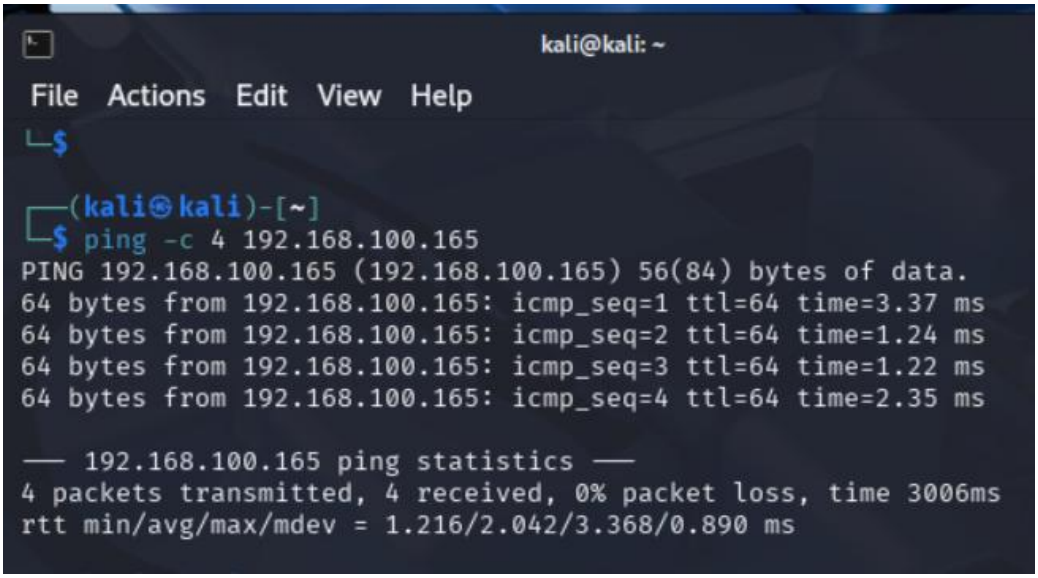


Figura 3. Comprobación de la conectividad con MASTER. Los paquetes enviados desde Kali reciben respuesta del servidor MASTER.

Comando utilizado

ssh root@192.168.100.165

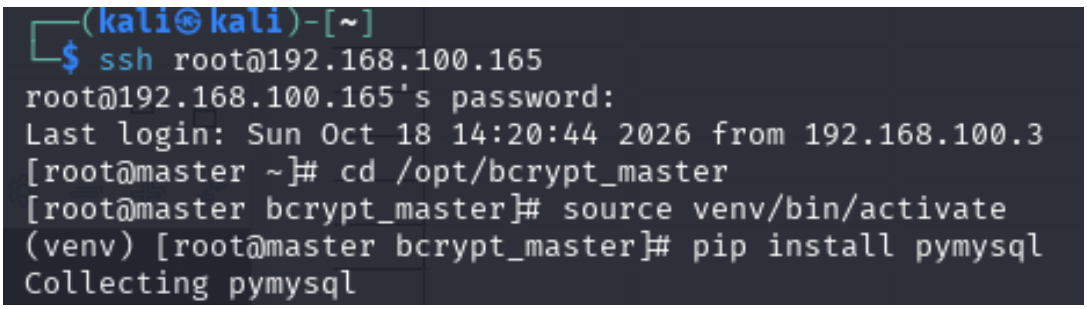


Figura 4. Acceso al servidor MASTER mediante SSH. La terminal de Kali muestra el inicio de sesión y la entrada al proyecto en MASTER.

3.2.2. Creación de la base inicial en SQLite

Dentro de MASTER utilicé SQLite para preparar la primera versión de la tabla usuarios. Definí un identificador, un nombre de usuario único, el campo password_hash y la fecha de registro. Con esta estructura pude comprobar cómo se guardaban los datos antes de instalar MariaDB.

Comando utilizado
sqlite3 usuarios.db

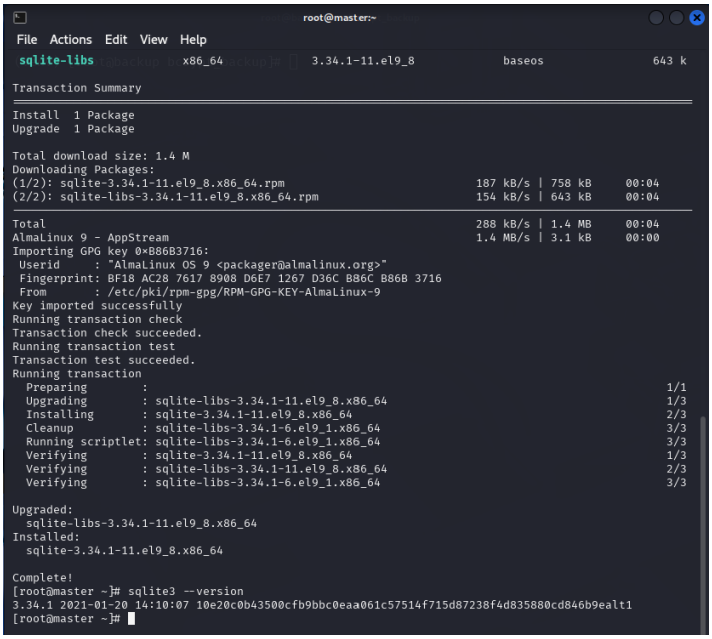


Figura 5. Instalación y apertura de SQLite en MASTER. La consola muestra la preparación de SQLite para crear la primera base de usuarios.

Comando utilizado
CREATE TABLE IF NOT EXISTS usuarios (id INTEGER PRIMARY KEY AUTOINCREMENT, usuario TEXT NOT NULL UNIQUE, password_hash TEXT NOT NULL, fecha_registro TEXT DEFAULT CURRENT_TIMESTAMP);



Figura 6. Estructura de la tabla usuarios en SQLite. La consulta del esquema permite comprobar los campos creados.

3.2.3. Preparación del entorno de Flask y bcrypt

Preparé el proyecto dentro de /opt/bcrypt_master, activé su entorno virtual e instalé Flask y bcrypt. Flask se utilizó para atender las solicitudes de la página web, mientras que bcrypt permitió generar y comprobar los hashes de las contraseñas.

Comando utilizado
python -m pip install flask bcrypt

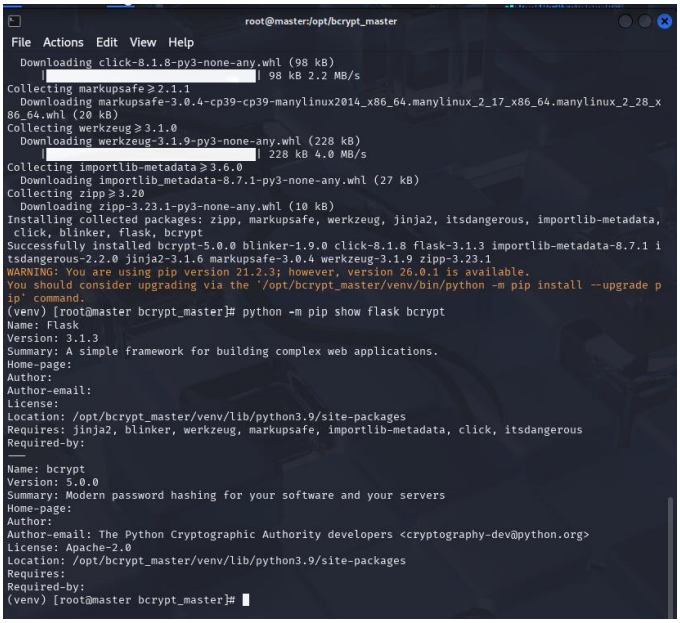


Figura 7. Instalación de Flask y bcrypt en MASTER. La terminal muestra las bibliotecas disponibles dentro del entorno virtual del proyecto.

3.2.4. Organización del proyecto y conexión inicial

Separé los formularios HTML en la carpeta templates y el archivo CSS en static. También edité app.py para iniciar Flask y consultar usuarios.db. Esta organización facilitó los cambios posteriores, porque pude mantener el diseño de la página cuando sustituí SQLite por MariaDB.

Comando utilizado
mkdir -p templates static

```
(venv) [root@master bcrypt_master]# cd /opt/bcrypt_master
(venv) [root@master bcrypt_master]# pwd
/opt/bcrypt_master
(venv) [root@master bcrypt_master]# mkdir -p templates static
(venv) [root@master bcrypt_master]# ls -la
total 16
drwxr-xr-x. 5 root root    68 Oct 18 00:09 .
drwxr-xr-x. 3 root root    27 Oct 17 23:52 ..
drwxr-xr-x. 2 root root     6 Oct 18 00:09 static
drwxr-xr-x. 2 root root     6 Oct 18 00:09 templates
-rw-r--r--. 1 root root 16384 Oct 17 23:53 usuarios.db
drwxr-xr-x. 5 root root    74 Oct 18 00:00 venv
(venv) [root@master bcrypt_master]#
```

Figura 8. Carpetas de la aplicación en MASTER. Se observan los directorios templates y static utilizados para organizar el proyecto.

Comando utilizado
nano app.py

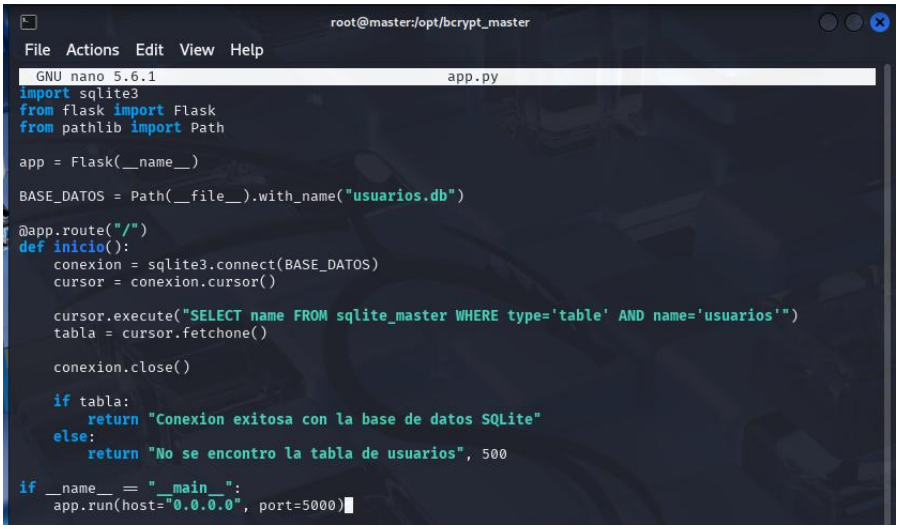


Figura 9. Primera conexión de Flask con SQLite. En app.py se muestra la referencia al archivo usuarios.db durante la fase inicial.

3.2.5. Ejecución de Flask y verificación del hash

Ejecuté la aplicación en el puerto 5000. Luego hice un registro de prueba y consulté la tabla usuarios de SQLite. El campo password_hash contenía una cadena generada por bcrypt en lugar de la contraseña que se escribió en el formulario.

Comando utilizado
python app.py

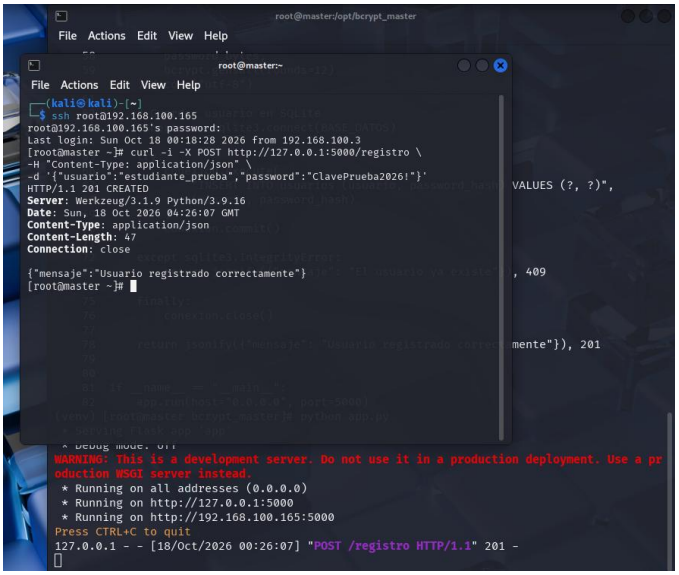


Figura 10. Servidor Flask ejecutándose en MASTER. La terminal muestra la aplicación disponible desde la IP de MASTER en el puerto 5000.

Comando utilizado
SELECT id, usuario, password_hash FROM usuarios;

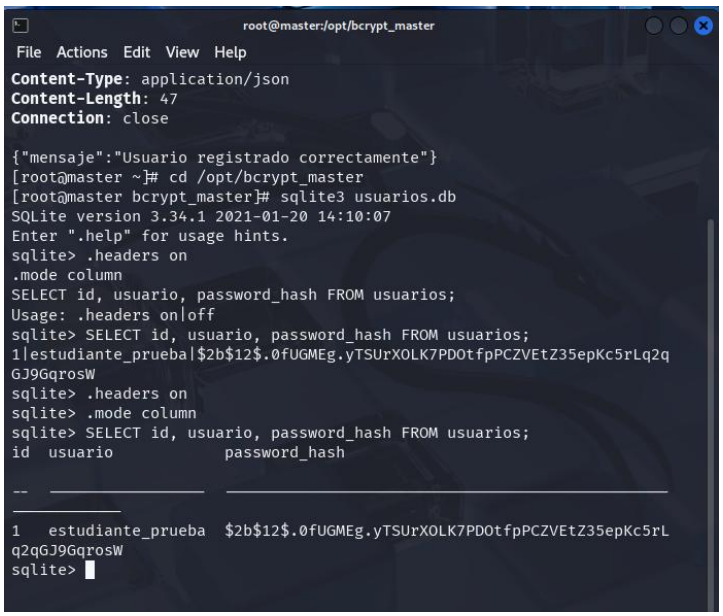


Figura 11. Comprobación del hash bcrypt en SQLite. La tabla contiene el usuario de prueba y el valor protegido de su contraseña.

3.3. Desarrollo y pruebas de las pantallas del Portal Estudiantil

3.3.1. Formulario de registro y estilos de la página

Preparé el formulario de registro en HTML y personalicé la apariencia con CSS. Incluí los campos de usuario, contraseña y confirmación. También agregué mensajes para que la persona identifique si el registro se completó correctamente.

Comando utilizado
nano templates/index.html

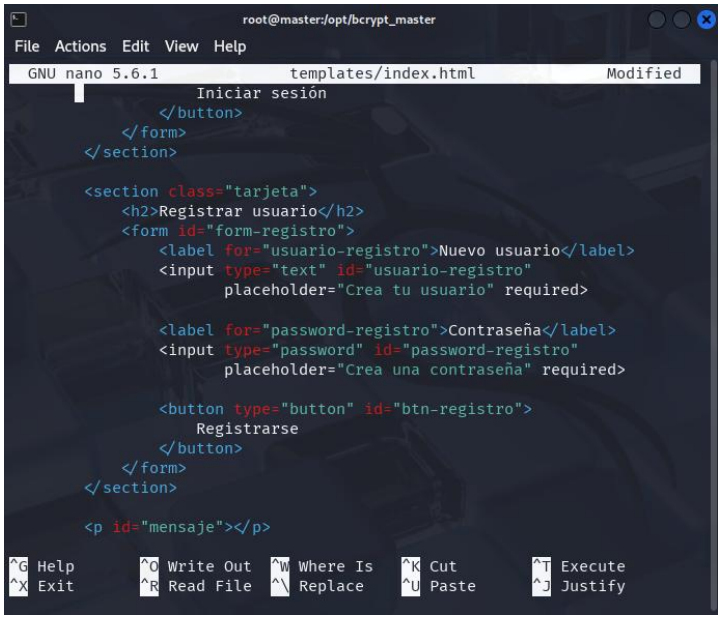


Figura 12. Código HTML del registro. Se aprecian los campos y botones del formulario de creación de usuarios.

Comando utilizado
nano static/estilos.css

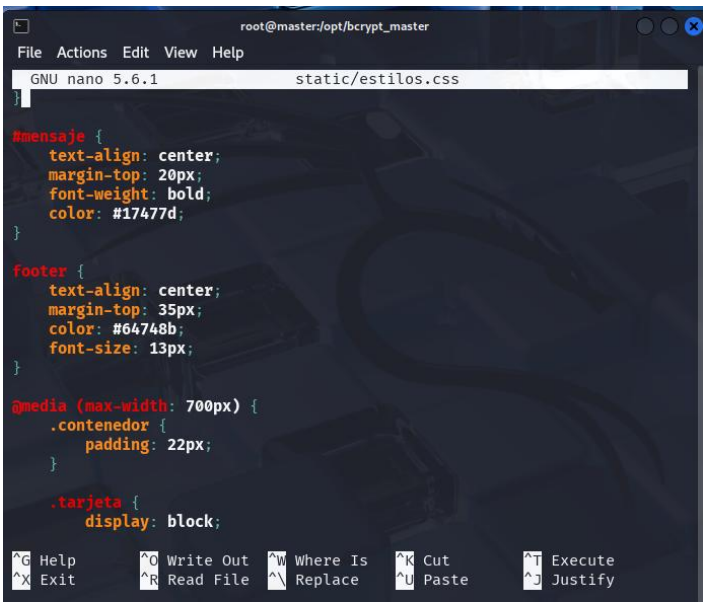


Figura 13. Estilos del Portal Estudiantil. La captura muestra reglas CSS utilizadas para presentar de manera ordenada la página.

3.3.2. Prueba del registro desde Firefox

Abrí la dirección del servidor MASTER desde Firefox en Kali y realicé registros con datos de prueba. Cuando las validaciones se cumplieron, el formulario mostró el mensaje de confirmación. Antes de guardar la cuenta, Flask aplicó bcrypt a la contraseña.



Figura 14. Registro desde el navegador. La pantalla muestra el formulario del Portal Estudiantil y el mensaje de registro exitoso.

3.3.3. Inicio de sesión y control de acceso

También creé la pantalla para iniciar sesión. La aplicación consulta el usuario registrado y utiliza bcrypt.checkpw para comparar la contraseña ingresada con el hash almacenado. Probé un acceso correcto y otro con una contraseña incorrecta para comprobar ambos resultados.

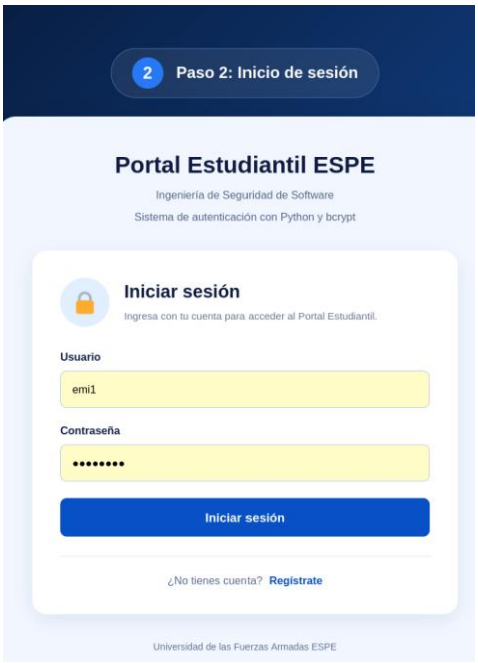


Figura 15. Formulario de inicio de sesión. El sistema solicita las credenciales para comprobar el acceso.

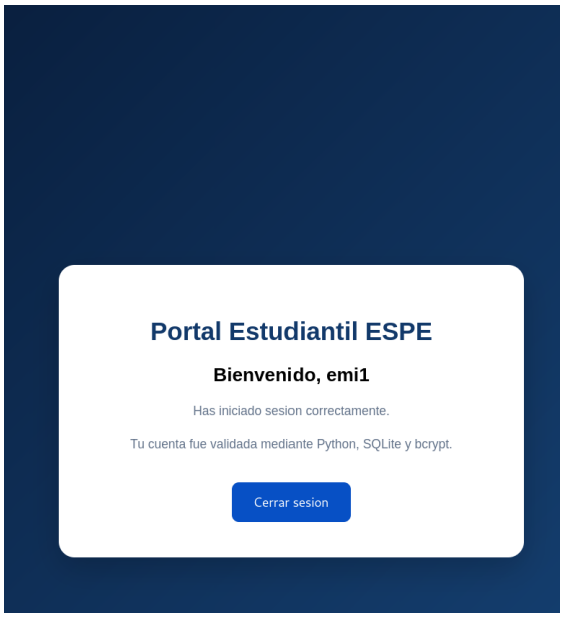


Figura 16. Ingreso permitido al panel. Con credenciales válidas se visualiza la pantalla de bienvenida del Portal Estudiantil.

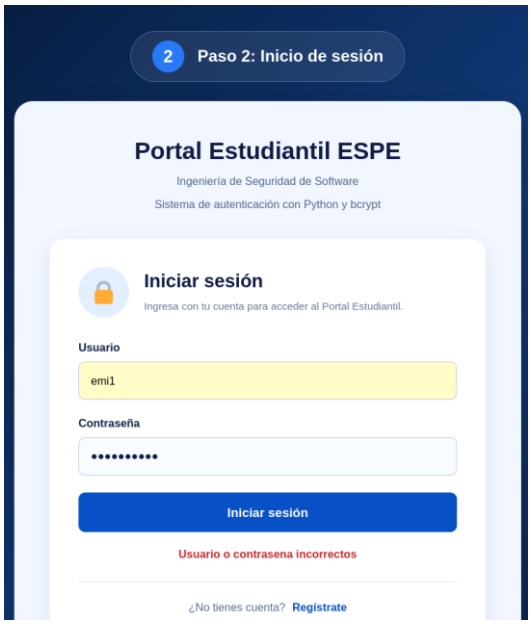


Figura 17. Rechazo de credenciales incorrectas. Cuando la contraseña no coincide, la página muestra el mensaje de error y evita el acceso.

3.4. Implementación final de MariaDB en MASTER

3.4.1. Creación de la nueva base de datos

La primera etapa con SQLite funcionó, pero para revisar los registros desde Kali necesitaba copiar el archivo usuarios.db. Por eso configuré MariaDB directamente en MASTER y creé una tabla con los mismos datos esenciales. La aplicación empezó a registrar nuevos usuarios en MariaDB una vez actualizada la conexión de Flask; la copia anterior de SQLite se conservó como respaldo.

Comando utilizado
CREATE DATABASE IF NOT EXISTS portal_estudiantil CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
Comando utilizado
CREATE TABLE IF NOT EXISTS usuarios (id INT AUTO_INCREMENT PRIMARY KEY, usuario VARCHAR(40) NOT NULL UNIQUE, password_hash VARCHAR(255) NOT NULL, fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP);

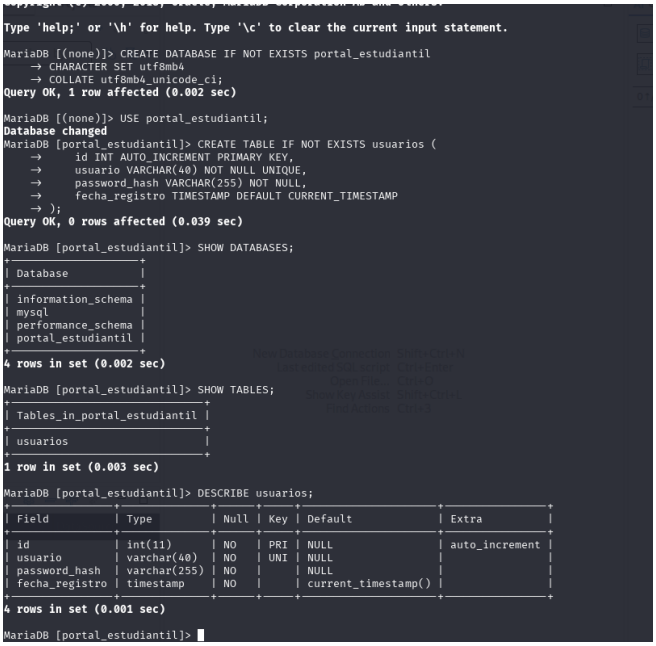


Figura 18. Creación de MariaDB y de la tabla usuarios. En MASTER se comprueba la existencia de portal_estudiantil y los campos de la tabla.

3.4.2. Asignación de permisos a la aplicación

Para no conectar Flask con una cuenta administradora creé el usuario portal_app y le asigné permisos de lectura e inserción sobre la tabla. La cuenta portal_visor se utilizó por separado para consultar los registros desde DBeaver.

Comando utilizado
GRANT SELECT, INSERT ON portal_estudiantil.usuarios TO 'portal_app'@'localhost';

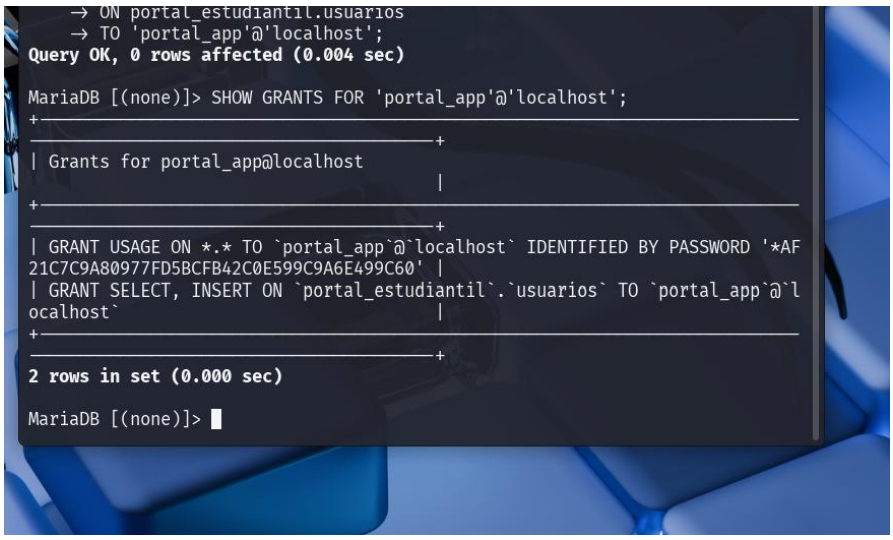


Figura 19. Verificación de permisos de portal_app. La consulta muestra SELECT e INSERT como operaciones permitidas para la aplicación.

3.4.3. Conexión de Python con MariaDB

Dentro del entorno virtual instalé PyMySQL y cambié en app.py la función de conexión y las consultas de registro e inicio de sesión. Conservé bcrypt para el tratamiento de contraseñas. Después comprobé que Flask podía trabajar con MariaDB y que los registros nuevos aparecían en DBeaver.

Comando utilizado
pip install pymysql



Figura 20. Instalación de PyMySQL en MASTER. La terminal confirma la instalación del conector que utiliza Flask para acceder a MariaDB.

3.4.4. Comprobación final de la aplicación

Realicé un nuevo inicio de sesión con una cuenta de prueba creada en MariaDB y abrí el panel privado. De forma complementaria, confirmé los registros en DBeaver, como se presentó al inicio del desarrollo. La columna password_hash mostraba hashes bcrypt, no las contraseñas originales.

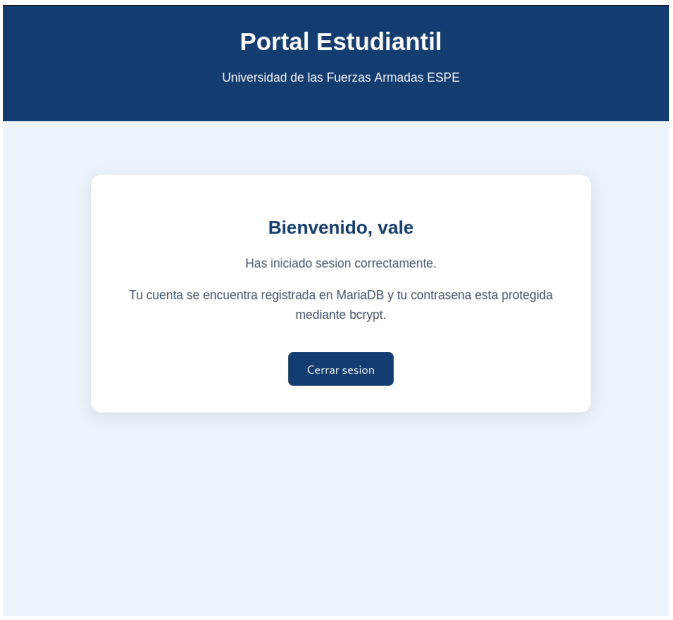


Figura 21. Panel Estudiantil con la base MariaDB. La pantalla confirma el acceso después de validar una cuenta almacenada en la base final.

3.5. Preparación de la entrega y evidencias en GitHub

3.5.1. Exportación del archivo SQL

Generé portal_estudiantil.sql como respaldo de la estructura de la tabla para incluirlo con el código. Inicialmente lo exporté sin datos. Más adelante se preparó una exportación con registros de laboratorio para mostrar hashes de prueba. El archivo SQL es una copia del momento de exportación; los registros nuevos aparecen directamente en MariaDB, no se agregan automáticamente al archivo.

Comando utilizado
mariadb-dump --no-data portal_estudiantil usuarios > portal_estudiantil.sql

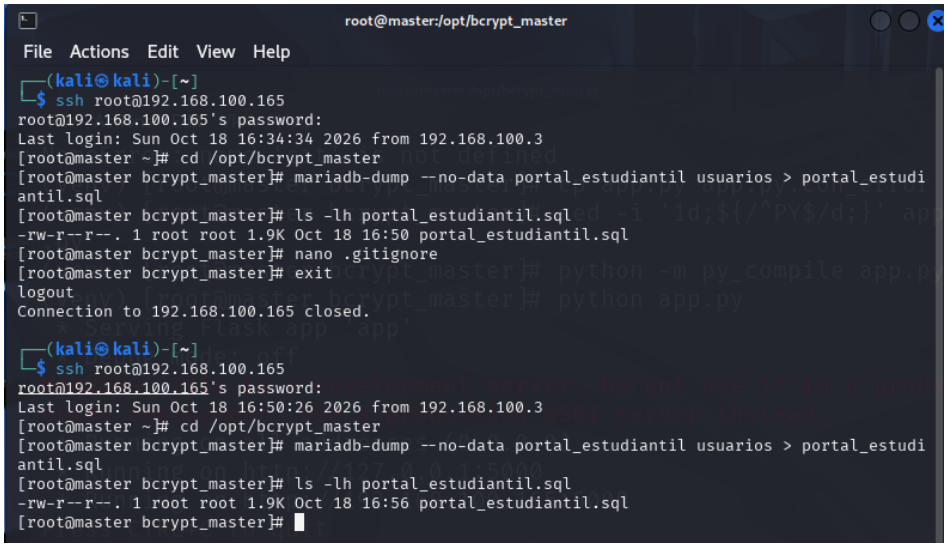


Figura 22. Primera exportación de portal_estudiantil.sql. La terminal muestra la generación inicial de la estructura SQL para la entrega.

3.5.2. Organización de los archivos del repositorio

Para preparar la entrega utilicé Git en MASTER y seleccioné el código de Python, la carpeta templates, los estilos de static y el archivo SQL. Excluí el entorno virtual, los respaldos y los archivos privados. Finalmente preparé la subida al repositorio de GitHub.

Comando utilizado
git add app.py templates/ static/ portal_estudiantil.sql .gitignore

```
[root@master bcrypt_master]# git commit -m "Portal Estudiantil con Flask, bcrypt y MariaDB"
[main (root-commit) d429837] Portal Estudiantil con Flask, bcrypt y MariaDB
 6 files changed, 1151 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 app.py
 create mode 100644 portal_estudiantil.sql
 create mode 100644 static/estilos.css
 create mode 100644 templates/index.html
 create mode 100644 templates/login.html
```

Figura 23. Preparación del proyecto con Git. Se observan los comandos utilizados para organizar los archivos antes de subirlos.

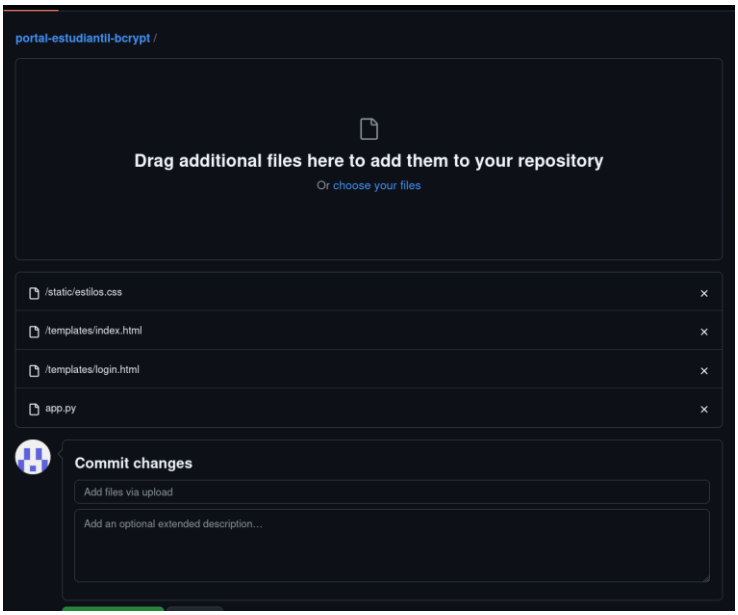


Figura 24. Archivos seleccionados para GitHub. La página de carga muestra el código Python, los formularios, el CSS y el archivo SQL listos para confirmar.

Repositorio del proyecto: https://github.com/emily-cmd/portal-estudiantil-bcrypt

4. Resultados obtenidos

El sistema de registro e inicio de sesión quedó funcionando en MASTER con Flask y bcrypt. Primero se hicieron pruebas con SQLite y después se configuró MariaDB para almacenar los nuevos registros. En las capturas se observa que el Portal Estudiantil confirma los registros válidos, permite entrar al panel con credenciales correctas y rechaza las contraseñas que no coinciden.

La consulta gráfica de DBeaver mostró cuatro usuarios de prueba y sus hashes bcrypt en MariaDB. Esta verificación fue importante porque pude comprobar desde Kali que la aplicación y la base de datos del servidor MASTER estaban relacionadas, sin tener que utilizar una copia separada de SQLite.

Las capturas de GitHub documentan la preparación y selección de archivos para la entrega. No se muestra en ellas una confirmación final de publicación del repositorio. La práctica no incluyó la configuración de una réplica de MariaDB ni el despliegue de la aplicación en BACKUP.

5. Conclusión

Con esta práctica pude comprender cómo se relacionan una aplicación web y su base de datos durante el registro y el inicio de sesión. Lo más importante fue comprobar que bcrypt permite verificar contraseñas sin guardarlas directamente en la tabla de usuarios. Las pruebas con credenciales correctas e incorrectas me ayudaron a confirmar que la validación funcionaba.

Trabajar primero con SQLite me permitió entender las consultas desde la terminal. Después, al conectar Flask con MariaDB y visualizar los datos desde DBeaver, pude comprobar de una forma más clara el resultado de cada registro. También reforcé el uso de SSH, los entornos virtuales de Python y la organización de un proyecto web para presentarlo con evidencias.

6. Recomendación

Recomiendo conservar el uso de bcrypt y las validaciones del lado del servidor. También es importante utilizar contraseñas de prueba durante las prácticas, proteger la clave de conexión a MariaDB y asignar a cada usuario de base de datos únicamente los permisos necesarios. Al publicar el proyecto en GitHub, se deben excluir los archivos privados, las bases locales y cualquier credencial.

Para una implementación fuera del laboratorio convendría incorporar HTTPS, un servidor web adecuado para producción, controles de acceso más completos y copias de seguridad periódicas. En caso de necesitar alta disponibilidad, la configuración y sincronización del servidor BACKUP debería realizarse como una práctica adicional.

7. Referencias bibliográficas

- GitHub. (s. f.). Agregar un archivo a un repositorio. Documentación de GitHub. https://docs.github.com/es/repositories/working-with-files/managing-files/adding-a-file-to-a-repository
- MariaDB Foundation. (s. f.). MariaDB en resumen. https://mariadb.org/es/
- Pallets Projects. (s. f.). Inicio rápido. Documentación de Flask. https://flask.palletsprojects.com/es/stable/quickstart/
- Python Software Foundation. (s. f.-a). Entornos virtuales y paquetes. https://docs.python.org/es/3/tutorial/venv.html
- Python Software Foundation. (s. f.-b). sqlite3 — Interfaz DB-API 2.0 para bases de datos SQLite. https://docs.python.org/es/3/library/sqlite3.html